You said:

Assignment - Pyth...
Document

i want to submit this project so give me how i do step by step give me evry step to create this oroject.

Khushi Walje khushi@tradexa.co.in

Attachments2:07 PM (6 hours ago)

---

to me image

Hello Jui,

Congratulations! Your profile has been shortlisted for the Python/Django Developer Intern opportunity.

Please complete and submit the assignment within 24 hours of receiving this email.

Assignment Deliverables:

- GitHub Repository Link
- README.md
- AI_USAGE.md (including AI tools used, prompts, accepted/rejected outputs, mistakes identified, and verification steps)
- Exported chat transcript attached and included in the repository
- Test Cases
- Chat Transcript File
- Test Run Output (either a GitHub Actions link or a TEST_OUTPUT.md file containing terminal output)

Please share all the required deliverables in a single ZIP file for ease of

review.

Job Requirements Confirmation:

Additionally, please confirm your alignment with the following internship requirements:

- Location: Are you open to relocating to Koregaon Park, Pune, for this on-site role?
- Availability: What is your earliest availability to join?
- Stipend: Are you comfortable with a monthly stipend of INR 13,000 - 15,100?

We look forward to your submission.

--\

Thanks and Regards,
Khushi Walje,
Human Resource Department,
Tradexa Technologies Pvt Ltd.

ChatGPT said:

Jui, I read your uploaded **Python/Django Hiring Assignment: AI-Assisted Box Selection System**. I'll guide you step by step, from creating the Django project in VS Code to testing it, uploading it to GitHub, and preparing the ZIP file for Tradexa Technologies.

Your assignment is to build a system that recommends the most suitable shipping box for an ecommerce order based on product dimensions, product weight, box dimensions, box weight capacity, and box cost. Assignment - Python_Django Developer _V1 (1).docx

Important: Your assignment specifically says that the chat transcript and the "What did you learn?" section must be written by you, not generated by AI. I can help you understand the project, build and test the application, and organize the other deliverables, but you should create those two items yourself in your own words.

1. Understand what you need to build



Step 1 — Products

Store product name, length, width, height, and weight. Example: a headphone box.



Step 2 — Shipping boxes

Store each box's internal length, width, height, maximum weight capacity, and cost.



Step 3 — Box recommendation

Check whether the product fits inside a box and whether the box can carry its weight. Among eligible boxes, recommend the lowest-cost suitable option.

For the first version, we'll assume **one product per order**, allow the product to rotate into any orientation, and use its actual weight against the box's maximum weight capacity. We can extend the logic to orders containing multiple products if the assignment's detailed requirements call for it.

2. Technologies to use

Technology	Purpose
Python	Recommendation logic
Django	Backend and web application
SQLite	Store products and boxes
HTML/CSS	User interface

Django TestCase	Automated tests
Git/GitHub	Version control and submission

3. Create your Django project in VS Code

We will use Windows PowerShell, just like your previous Django projects.

Step 1 — Create the project folder

Open VS Code, open the terminal using **Terminal** → **New Terminal**, and run:

```
cd F:\
mkdir box_selection_project
cd box_selection_project
code .
```

If `F:` is not available on your computer, use a folder on your `C:` drive instead.

Step 2 — Create a virtual environment

Run each command one by one:

```
py -m venv venv
```

Activate the environment:

```
.\venv\Scripts\Activate.ps1
```

Your terminal should begin with `(venv)`.

Install Django:

```
python -m pip install --upgrade pip
pip install "Django>=5.2,<6.0"
```

Verify the installation:

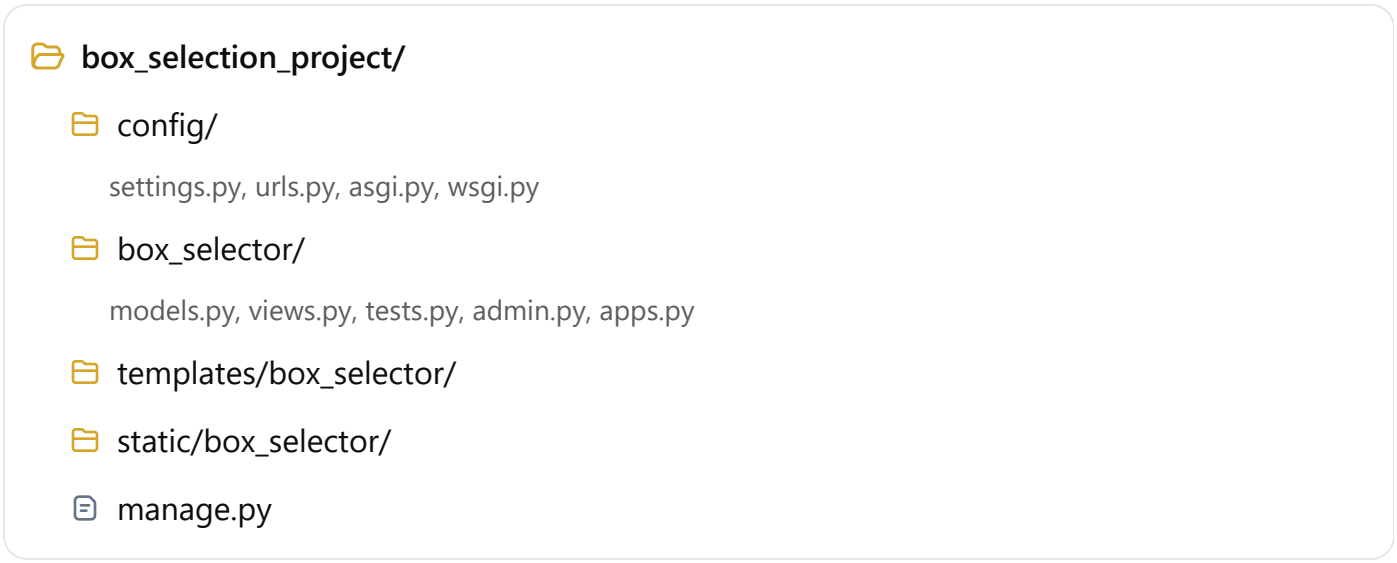
```
python -m django --version
```

Step 3 — Create the Django project and app

Run:

```
django-admin startproject config .
python manage.py startapp box_selector
```

Your folder structure will start like this:



Step 4 — Register your app

Open `config/settings.py`.

Find `INSTALLED_APPS` and add `'box_selector'` to the list:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
```

```
    'box_selector',
]
```

Save the file.

Step 5 — Create your database models

Open `box_selector/models.py` and replace its contents with the following starter implementation.

```
from django.db import models
from django.core.validators import MinValueValidator


class Product(models.Model):
    name = models.CharField(max_length=150, unique=True)
    length = models.DecimalField(
        max_digits=8, decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )
    width = models.DecimalField(
        max_digits=8, decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )
    height = models.DecimalField(
        max_digits=8, decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )
    weight = models.DecimalField(
        max_digits=8, decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )

    def __str__(self):
        return self.name


class ShippingBox(models.Model):
    name = models.CharField(max_length=100, unique=True)
    length = models.DecimalField(
        max_digits=8, decimal_places=2,
```

```
        validators=[MinValueValidator(0.01)]
    )
    width = models.DecimalField(
        max_digits=8, decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )
    height = models.DecimalField(
        max_digits=8, decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )
    max_weight = models.DecimalField(
        max_digits=8, decimal_places=2,
        validators=[MinValueValidator(0.01)]
    )
    cost = models.DecimalField(
        max_digits=8, decimal_places=2,
        validators=[MinValueValidator(0)]
    )

    def __str__(self):
        return self.name
```

Important: This starter model uses the same dimension unit for products and boxes, such as centimetres, and the same weight unit, such as kilograms. Make these units clear in the user interface and README.

The two models store the information needed for the recommendation engine. We will next connect them to the admin panel and build the actual box-selection logic.

Step 6 — Create database tables

In the terminal, run:

```
python manage.py makemigrations
python manage.py migrate
```

Expected result: Django creates the database tables without errors.

Step 7 — Register products and boxes in the admin panel

Open `box_selector/admin.py`:

```
from django.contrib import admin
from .models import Product, ShippingBox


@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
    list_display = (
        'id', 'name', 'length', 'width',
        'height', 'weight'
    )
    search_fields = ('name',)


@admin.register(ShippingBox)
class ShippingBoxAdmin(admin.ModelAdmin):
    list_display = (
        'id', 'name', 'length', 'width',
        'height', 'max_weight', 'cost'
    )
    search_fields = ('name',)
```

Create an administrator:

```
python manage.py createsuperuser
```

Enter the requested username, email and password.

Start the server:

```
python manage.py runserver
```

Open these URLs in your browser:

- Admin panel: `http://127.0.0.1:8000/admin/`
- Main application: `http://127.0.0.1:8000/` — we will configure this route in the next steps.

Log in to the admin panel and add some sample products and boxes. For example, you can enter a product with dimensions `10 × 8 × 5 cm`, weight `0.5 kg`, and boxes with different dimensions, capacities and costs.

4. Build the box recommendation algorithm

This is the most important part of the assignment. Your interviewer may ask you to explain why your system selects a particular box.

Step 8 — Create the recommendation logic

Create a new file named `box_selector/recommendation.py`.

Use this implementation as a starting point, and make sure you can explain and test it yourself.

```
from decimal import Decimal


def fits_inside(product, box):
    """
    Return True when the product fits in the box
    in at least one of its six possible orientations.
    Dimensions must use the same unit.
    """

    product_dimensions = sorted([
        product.length,
        product.width,
        product.height,
    ])

    box_dimensions = sorted([
        box.length,
        box.width,
        box.height,
    ])

    dimensions_fit = all(
        p <= b
        for p, b in zip(product_dimensions, box_dimensions)
    )
```

```
    weight_fits = product.weight <= box.max_weight

    return dimensions_fit and weight_fits

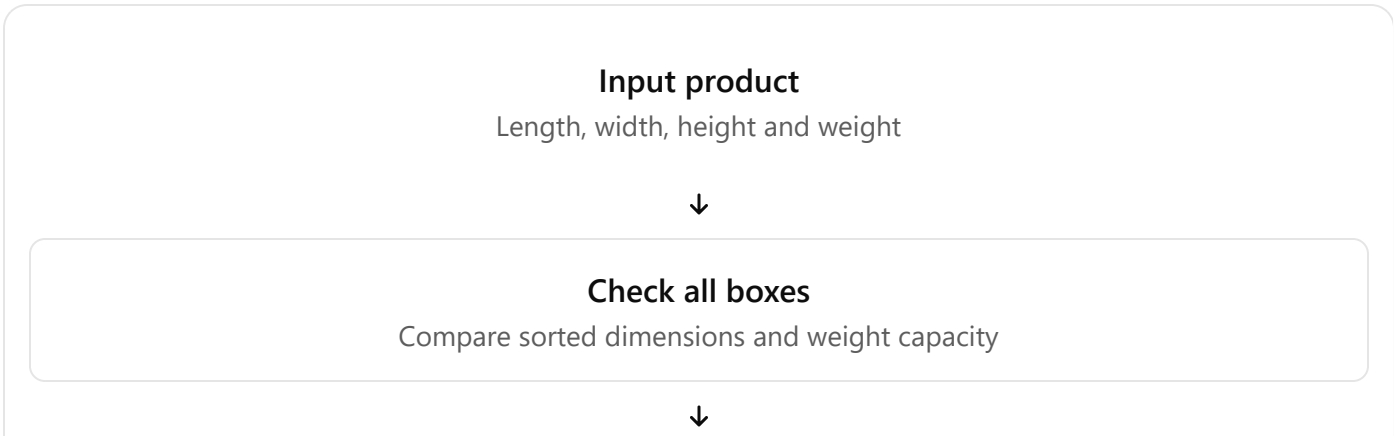

def recommend_box(product, boxes):
    """
    Return the lowest-cost suitable box.
    Return None when no box is suitable.
    """

    suitable_boxes = [
        box for box in boxes
        if fits_inside(product, box)
    ]

    if not suitable_boxes:
        return None

    return min(
        suitable_boxes,
        key=lambda box: (box.cost, box.pk or 0)
    )
```

How the algorithm works



Suitable boxes

Choose the lowest-cost option.

No suitable box

Return no recommendation.

Sorting the dimensions allows the algorithm to consider rotations. For example, a product measuring `10 × 20 × 5` can fit inside a box measuring `20 × 10 × 8`, provided its weight also fits.

Limitation to document: This version assumes one product per order. It does not calculate how to arrange several products inside the same box, and it does not include protective packaging clearance.

5. Connect the algorithm to the Django website

Step 9 — Create the view

Open `box_selector/views.py` and add:

```
from django.shortcuts import render
from .models import Product, ShippingBox
from .recommendation import recommend_box


def home(request):
    products = Product.objects.all().order_by('name')

    selected_product = None
    recommended_box = None
    checked = False
    error = None

    if request.method == 'POST':
        checked = True
        product_id = request.POST.get('product_id')

        try:
            selected_product = Product.objects.get(
                pk=product_id
            )
```

```
            recommended_box = recommend_box(
                selected_product,
                ShippingBox.objects.all()
            )

            if recommended_box is None:
                error = (
                    'No suitable box is available '
                    'for this product.'
                )

        except (Product.DoesNotExist, ValueError, TypeError):
            error = 'Please select a valid product.'

    return render(request, 'box_selector/home.html', {
        'products': products,
        'selected_product': selected_product,
        'recommended_box': recommended_box,
        'checked': checked,
        'error': error,
    })
```

This view reads the selected product, calls the recommendation function and sends the result to the HTML template.

Step 10 — Create the URL files

Create `box_selector/urls.py`:

```
from django.urls import path
from . import views

app_name = 'box_selector'

urlpatterns = [
    path('', views.home, name='home'),
]
```

Now open `config/urls.py` and replace its contents:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('box_selector.urls')),
]
```

The home page is now connected to the view.

Step 11 — Create the HTML template

Create these folders if they do not exist:

```
box_selector/
    templates/
        box_selector/
            home.html
```

Open `box_selector/templates/box_selector/home.html` and add:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AI-Assisted Box Selection</title>
</head>
<body>
    <h1>AI-Assisted Box Selection System</h1>
    <p>Select a product to find the lowest-cost suitable box.</p>

    {% if products %}
        <form method="post">
            {% csrf_token %}

            <label for="product_id">Product:</label>
            <select id="product_id" name="product_id" required>
                {% for product in products %}
```

```
                    <option value="{{ product.pk }}"
                        {% if selected_product.pk == product.pk %}
                            selected
                        {% endif %}>
                        {{ product.name }}
                    </option>
                {% endfor %}
            </select>

            <button type="submit">Recommend Box</button>
        </form>
    {% else %}
        <p>No products available. Add products in the admin panel.</p>
    {% endif %}

    {% if checked %}
        {% if error %}
            <h2>No recommendation</h2>
            <p>{{ error }}</p>
        {% elif recommended_box %}
            <h2>Recommended Box</h2>
            <p>Product: {{ selected_product.name }}</p>
            <p>Box: {{ recommended_box.name }}</p>
            <p>Dimensions:
                {{ recommended_box.length }} ×
                {{ recommended_box.width }} ×
                {{ recommended_box.height }}
            </p>
            <p>Maximum weight: {{ recommended_box.max_weight }} kg</p>
            <p>Box cost: ₹{{ recommended_box.cost }}</p>
        {% endif %}
    {% endif %}

    <p><a href="/admin/">Manage products and boxes</a></p>
</body>
</html>
```

This is the initial functional interface. After it works, you can improve the design using a separate CSS file under `static/box_selector/` and load it using Django's `{% load static %}` tag.

Step 12 — Run the website

Run:

```
python manage.py check
python manage.py runserver
```

Open `http://127.0.0.1:8000/`.

Test the following workflow:

1. Visit `/admin/`.
2. Add at least three products and three shipping boxes.
3. Return to the home page.
4. Select a product and click **Recommend Box**.
5. Confirm that the result is the cheapest box that satisfies both dimension and weight constraints.
6. Try a product that cannot fit any available box.

At this point, you have a basic working Django application. The next stage is essential for a hiring assignment: automated testing, proper documentation, and a reproducible submission.

6. Write automated test cases

Step 13 — Create recommendation tests

Open `box_selector/tests.py` and replace its contents with:

```
from django.test import TestCase
from .models import Product, ShippingBox
from .recommendation import fits_inside, recommend_box


class BoxRecommendationTests(TestCase):

    def setUp(self):
        self.product = Product.objects.create(
            name="Headphones",
            length=10,
            width=8,
```

```
            height=5,
            weight=0.5,
        )

        self.cheap_box = ShippingBox.objects.create(
            name="Small Box",
            length=12,
            width=10,
            height=7,
            max_weight=1,
            cost=20,
        )

        self.expensive_box = ShippingBox.objects.create(
            name="Large Box",
            length=20,
            width=15,
            height=10,
            max_weight=5,
            cost=50,
        )

    def test_product_fits_inside_box(self):
        self.assertTrue(fits_inside(
            self.product, self.cheap_box
        ))

    def test_selects_lowest_cost_suitable_box(self):
        result = recommend_box(
            self.product,
            ShippingBox.objects.all()
        )
        self.assertEqual(result, self.cheap_box)

    def test_rejects_box_with_insufficient_weight_capacity(self):
        self.cheap_box.max_weight = 0.2
        self.cheap_box.save()

        self.assertFalse(fits_inside(
            self.product, self.cheap_box
        ))
```

```
    def test_returns_none_when_no_box_fits(self):
        ShippingBox.objects.all().delete()

        result = recommend_box(
            self.product,
            ShippingBox.objects.all()
        )
        self.assertIsNone(result)

    def test_product_can_fit_after_rotation(self):
        self.product.length = 10
        self.product.width = 20
        self.product.height = 5
        self.product.save()

        rotated_box = ShippingBox.objects.create(
            name="Rotated Box",
            length=20,
            width=10,
            height=8,
            max_weight=2,
            cost=30,
        )

        self.assertTrue(fits_inside(
            self.product, rotated_box
        ))

    def test_exact_dimension_and_weight_limits(self):
        exact_box = ShippingBox.objects.create(
            name="Exact Fit",
            length=10,
            width=8,
            height=5,
            max_weight=0.5,
            cost=10,
        )

        self.assertTrue(fits_inside(
            self.product, exact_box
        ))
```

These tests cover the main recommendation rules. Add further tests for invalid form submissions, missing products, duplicate names, and invalid dimensions as you improve the application.

Step 14 — Run the test suite

In the terminal:

```
python manage.py test box_selector
```

Django should display the test results and finish with `OK` if every test passes.

Do not write that the tests passed in your submission until you have actually run them. If a test fails, read the traceback, correct the code, and rerun the suite.

7. Prepare the required deliverables

Your recruiter listed a specific set of files. Keep them in the project root so the reviewer can find them easily. Assignment - Python_Django Developer _V1 (1).docx

Step 15 — Create `README.md`

Create `README.md` in the main project folder.

Start with this documentation outline and fill in details based on the application you actually built and tested:

```
# AI-Assisted Box Selection System

## Overview
A Django application that recommends a suitable shipping
box for an ecommerce product based on dimensions, weight
capacity, and box cost.

## Features
- Product management through Django Admin
- Shipping box management through Django Admin
- Dimension and weight validation
- Lowest-cost suitable box recommendation
- Automated tests
```

```
## Technologies
- Python
- Django
- SQLite
- HTML and CSS
- Django TestCase

## Requirements
- Python 3.12 or another tested compatible version
- Dependencies listed in requirements.txt

## Setup
1. Clone the GitHub repository.
2. Create and activate a virtual environment.
3. Install dependencies:
   pip install -r requirements.txt
4. Apply database migrations:
   python manage.py migrate
5. Create an admin user:
   python manage.py createsuperuser
6. Start the server:
   python manage.py runserver

## Usage
1. Open /admin/ and add products and shipping boxes.
2. Open the home page.
3. Select a product and request a recommendation.
4. Review the recommended box and its cost.

## Recommendation Logic
Explain the dimension comparison, allowed rotations,
weight constraint, and cost-based selection.

## Assumptions and Limitations
- One product per order.
- Dimensions use the same unit.
- Product weight and box capacity use the same unit.
- Packaging clearance is not included.

## Testing
Run:
```

```
python manage.py test box_selector

## Project Structure
Describe the main Django files and folders.
```

Also create `requirements.txt` from the activated virtual environment:

```
pip freeze > requirements.txt
```

Review the file and ensure it lists the dependencies required by your project.

Step 16 — Create `AI_USAGE.md`

The recruiter specifically asks you to document which AI tools you used, prompts, accepted and rejected outputs, mistakes found, and verification steps. Assignment - Python_Django Developer _V1 (1).docx

Use a truthful record like this template. Complete the details yourself; do not claim that you rejected an output or discovered a mistake unless that actually happened.

```
# AI Usage Disclosure

## AI Tools Used
- Tool: ChatGPT
- Purpose: Understanding Django structure, algorithm design,
  test design, and documentation requirements.

## Prompts Used
Record the actual prompts submitted during development.
Include the date and the purpose of each prompt.

## Outputs Accepted
Record each suggestion that was used and explain why
it was appropriate for the assignment.

## Outputs Rejected or Modified
Record suggestions that were not used or that you changed.
Explain your reason for each decision.

## Mistakes Identified
```

```
Record actual errors, incorrect assumptions, or incomplete
suggestions you identified during implementation.

## Verification Steps
Record the commands you ran, tests you performed, and
results you personally observed.

## Developer Review
Explain which parts you reviewed, modified, understood,
and verified yourself.
```

Because you are using AI assistance for this assignment, disclose it honestly. Your final code and explanations should reflect what you understand and can defend during an interview.

Step 17 — Create `TEST_OUTPUT.md`

Run your test command and copy the **actual terminal output** into this file.

```
python manage.py test box_selector
```

Example structure:

```
# Test Run Output

## Command
python manage.py test box_selector

## Environment
Python version: [your actual version]
Django version: [your actual version]
Operating system: Windows

## Actual Terminal Output
[Paste the real terminal output here.]

## Result
[State the actual result: passed, failed, or partially passed.]
```

This file is only complete after you insert your actual output. Alternatively, you can configure GitHub Actions and submit a link to the successful workflow run.

8. Export your chat transcript correctly

Step 18 — Prepare the transcript yourself

This requirement is particularly important. The assignment explicitly says not to generate the exported chat transcript using AI and warns that doing so may lead to rejection.

Assignment - Python_Django Developer _V1 (1).docx

Follow these steps:

1. Open the actual conversation or conversations you used while developing the project.
2. Use ChatGPT's available export or print options to preserve your original conversation.
3. Save the original transcript without rewriting or fabricating messages.
4. Place the exported file in your project folder, for example:

```
chat_transcript/
    original_chat_transcript.html
```

Use the actual filename and format of your export. Do not create a synthetic conversation and label it as an original transcript.

Step 19 — Write “What did you learn?”

The assignment also asks you to describe what you learned, in your own words, without AI generating the answer. Write this section yourself after completing the project.

Think about questions such as:

- What did you learn about Django models and database migrations?
- How did you compare product dimensions and box capacities?
- What problem did you encounter during testing?
- How did you verify the cheapest suitable box was selected?

Do not copy an AI-written response into this section. Your own explanation will also help you prepare for the interview.

9. Upload the project to GitHub

Step 20 — Create `.gitignore`

In the root folder, create a file named `.gitignore`:

```
venv/
.venv/
__pycache__/
*.py[cod]
db.sqlite3
.env
.idea/
.vscode/
```

This prevents your virtual environment, local database and other unnecessary files from being committed. If your submission requires sample data, provide fixtures or a documented seed-data command rather than relying on your local database.

Step 21 — Initialize Git

Run these commands from your project root:

```
git init
git status
git add .
git commit -m "Build Django box selection system"
```

If Git asks you to configure your identity, use your own GitHub name and email.

Step 22 — Create a GitHub repository

GitHub

Create a new repository named `ai-assisted-box-selection`.

Choose Public if the assignment permits public repositories; otherwise use Private and confirm that the recruiter can access it.

Create a GitHub repository ↗

Do not initialize the remote repository with an extra README if you have already created one locally.

After creating the empty repository, copy its actual URL and run:

```
git branch -M main
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main
```

Replace `YOUR_GITHUB_REPOSITORY_URL` with the URL GitHub provides.

Open the repository in your browser and verify that the source code and documentation are visible.

10. Prepare the final ZIP file

Step 23 — Check your folder structure

Your finished project should resemble this:

- **ai-assisted-box-selection/**
  - config/
  - box_selector/
  - templates/ (inside the app)
  - static/ (inside the app, if used)
  - chat_transcript/
  - manage.py
  - requirements.txt
  - README.md
  - AI_USAGE.md
  - TEST_OUTPUT.md
  - .gitignore

Make sure `box_selector/` contains your `models.py`, `views.py`, `urls.py`, `recommendation.py`, `admin.py`, and `tests.py`.

Do not include: `venv/`, `__pycache__/`, unnecessary temporary files, or passwords and API keys.

Step 24 — Create the ZIP

Once the project has been tested and pushed to GitHub, use PowerShell from the parent directory:

```
Compress-Archive `
  -Path .\box_selection_project\* `
  -DestinationPath .\Tradexa_Box_Selection_Submission.zip
```

Adjust the folder name to match your actual project directory.

Review the ZIP before sending it. It should include the required source files, `README.md`, `AI_USAGE.md`, the original chat transcript, test cases and test output. The GitHub repository URL must also be included in the README or your submission email.

11. Final submission checklist

Submission readiness 0 of 12

- [ ] Django application runs without errors
- [ ] Products and shipping boxes can be added
- [ ] Correct recommendation is shown for suitable products
- [ ] No-fit and insufficient-weight cases work correctly
- [ ] Automated tests run and actual output is saved
- [ ] README.md is complete and accurate
- [ ] AI_USAGE.md truthfully documents AI assistance
- [ ] Original chat transcript is exported by me and included
- [ ] What I learned is written by me
- [ ] GitHub repository is accessible to the reviewer
- [ ] ZIP contains all required deliverables

- [ ] Secrets and virtual environment are excluded

12. Reply to the recruiter

You also need to confirm relocation, earliest joining date, and stipend acceptance. You can use this draft after checking that each answer is true for you.

Re: Python/Django Developer Intern – Assignment Submission Connect email

To

Re: Python/Django Developer Intern – Assignment Submission

Dear Khushi Ma'am,

Thank you for shortlisting my profile for the Python/Django Developer Intern opportunity.

I am working on the assignment and will submit the required deliverables in a single ZIP file, along with my GitHub repository link.

Please find my availability details below:

- **Relocation:** [Confirm whether you are willing to relocate to Koregaon Park, Pune.]
- **Earliest joining date:** [Enter your actual earliest joining date.]
- **Stipend:** [Confirm whether you are comfortable with the offered monthly stipend of ₹13,000–₹15,100.]

Thank you for this opportunity. I look forward to sharing my completed assignment.

Best regards,
Jui Garule

What I recommend you do now

Since you have only 24 hours from the email's receipt, focus first on the working application and its tests, then complete the documentation and submission packaging.

Start with Steps 1–7 in VS Code. If you encounter any error, send me the exact terminal error or a screenshot. We can fix it step by step before moving to the next stage.

You said:

where i create thisCreate a new file named `box_selector/recommendation.py` .

ChatGPT said:

Jui, you need to create `recommendation.py` **inside the** `box_selector` **folder** in your Django project using VS Code. Follow these steps.

Step 1: Open your project in VS Code

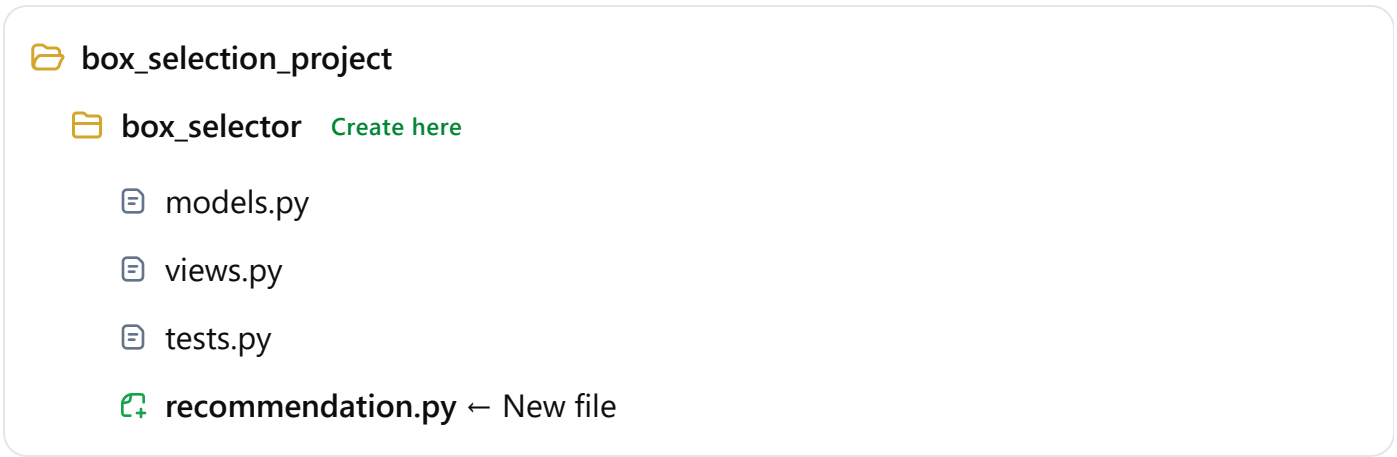
Your project folder should be:

```
F:\box_selection_project
```

In VS Code, look at the **Explorer panel on the left side**.

Step 2: Find the `box_selector` folder

Your folder structure should look like this:



Step 3: Create the file

1. Right-click the `box_selector` folder.
2. Click **New File**.
3. Type `recommendation.py`.
4. Press **Enter**.

Make sure the file is not named `recommendation.py.txt`.

Step 4: Paste the code

Open `recommendation.py` and paste this code:

```
def fits_inside(product, box):
    product_dimensions = sorted([
        product.length,
        product.width,
        product.height,
    ])

    box_dimensions = sorted([
        box.length,
        box.width,
        box.height,
    ])

    dimensions_fit = all(
        p <= b
        for p, b in zip(product_dimensions, box_dimensions)
    )

    weight_fits = product.weight <= box.max_weight

    return dimensions_fit and weight_fits


def recommend_box(product, boxes):
    suitable_boxes = [
        box for box in boxes
        if fits_inside(product, box)
    ]
```

```
    if not suitable_boxes:
        return None

    return min(
        suitable_boxes,
        key=lambda box: (box.cost, box.pk or 0)
    )
```

5. Press **Ctrl + S** to save.

That's it! Your new file is ready.

Next, you will connect this file to `views.py`, which calls the recommendation function when a user selects a product.

You said:

where i create template folder outside or inside of box_selector

ChatGPT said:

Create the `templates` **folder inside** `box_selector`, not outside it. 😊

Your folder structure should look like this in VS Code:

- **box_selection_project**
  - config
  - **box_selector**
    - **templates**
      - box_selector
        - home.html
    - models.py
    - views.py
    - recommendation.py
    - urls.py

manage.py

Step-by-step in VS Code

1. Right-click the `box_selector` folder.
2. Select **New Folder** and name it `templates`.
3. Right-click the new `templates` folder.
4. Select **New Folder** and name it `box_selector`.
5. Right-click the inner `box_selector` folder.
6. Select **New File** and name it `home.html`.
7. Paste the HTML code I gave you earlier into `home.html` and press **Ctrl + S**.

Important: The full path should be:

```
F:\box_selection_project\box_selector\templates\box_selector\home.html
```

After creating this file, tell me **"done"**, and we'll continue with the next step.

You said:

Pasted text(1).txt
Document

ChatGPT said:

Your error is clear! 😊 The main problem is:

```
AttributeError: module 'box_selector.views' has no attribute 'home'
```

This means Django is looking for a function named `home` in `box_selector/views.py`, but it cannot find it. Your `urls.py` is already trying to use `views.home`.

Step 1: Open `views.py`

In VS Code, open:

```
F:\box_Selection\box_selector\views.py
```

Replace the existing code with the following complete code:

```
from django.shortcuts import render
from .models import Product, ShippingBox
from .recommendation import recommend_box


def home(request):
    products = Product.objects.all().order_by("name")
    selected_product = None
    recommended_box = None
    checked = False
    error = None

    if request.method == "POST":
        checked = True
        product_id = request.POST.get("product_id")

        try:
            selected_product = Product.objects.get(pk=product_id)
            recommended_box = recommend_box(
                selected_product,
                ShippingBox.objects.all()
            )

            if recommended_box is None:
                error = "No suitable box is available for this product."

        except (Product.DoesNotExist, ValueError, TypeError):
            error = "Please select a valid product."

    return render(
        request,
        "box_selector/home.html",
        {
            "products": products,
            "selected_product": selected_product,
            "recommended_box": recommended_box,
            "checked": checked,
            "error": error,
        },
    )
```

Save the file using **Ctrl + S**.

Step 2: Check `urls.py`

Open `F:\box_Selection\box_selector\urls.py` and confirm it contains:

```
from django.urls import path
from . import views

app_name = "box_selector"

urlpatterns = [
    path("", views.home, name="home"),
]
```

Save it if you made any changes.

Step 3: Run the Django check

In your VS Code terminal, run:

```
python manage.py check
```

If it reports `System check identified no issues`, run:

```
python manage.py runserver
```

Then open `http://127.0.0.1:8000/` in your browser.

If you get another error, send me the new traceback and I'll help you fix the next issue. Don't change any other files yet.

You said:

```
Traceback (most recent call last):
  File "F:\box_Selection\manage.py", line 22, in <module>
    main()
    ~~~~^^
```

```
  File "F:\box_Selection\manage.py", line 18, in main
    execute_from_command_line(sys.argv)
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^
  File "F:\box_Selection\venv\Lib\site-
packages\django\core\management\__init__.py", line 443, in
execute_from_command_line
    utility.execute()
    ~~~~~~~~~~~~~~~^^
  File "F:\box_Selection\venv\Lib\site-
packages\django\core\management\__init__.py", line 437, in execute
    self.fetch_command(subcommand).run_from_argv(self.argv)

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^
^^^^^^^^^^
  File "F:\box_Selection\venv\Lib\site-
packages\django\core\management\base.py", line 422, in
run_from_argv
    self.execute(*args, **cmd_options)
    ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^
  File "F:\box_Selection\venv\Lib\site-
packages\django\core\management\base.py", line 466, in execute
    output = self.handle(*args, **options)
  File "F:\box_Selection\venv\Lib\site-
packages\django\core\management\commands\check.py", line 81, in
handle
    self.check(
    ~~~~~~~~~~^
        app_configs=app_configs,
        ^^^^^^^^^^^^^^^^^^^^^^^^
    ...<4 lines>...
        databases=options["databases"],
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "F:\box_Selection\venv\Lib\site-
packages\django\core\management\base.py", line 498, in check
    all_issues = checks.run_checks(
        app_configs=app_configs,
    ...<2 lines>...
        databases=databases,
    )
  File "F:\box_Selection\venv\Lib\site-
```

```
packages\django\core\checks\registry.py", line 99, in run_checks
    new_errors = check(app_configs=app_configs,
databases=databases)
  File "F:\box_Selection\venv\Lib\site-
packages\django\core\checks\urls.py", line 136, in
check_custom_error_handlers
    handler = resolver.resolve_error_handler(status_code)
  File "F:\box_Selection\venv\Lib\site-
packages\django\urls\resolvers.py", line 743, in resolve_error_handler
    callback = getattr(self.urlconf_module, "handler%s" % view_type,
None)
                    ^^^^^^^^^^^^^^^^^^^
  File "F:\box_Selection\venv\Lib\site-
packages\django\utils\functional.py", line 47, in __get__
    res = instance.__dict__[self.name] = self.func(instance)
                                  ~~~~~~~~~^^^^^^^^^^
  File "F:\box_Selection\venv\Lib\site-
packages\django\urls\resolvers.py", line 722, in urlconf_module
    return import_module(self.urlconf_name)
  File
"C:\Users\Mahajan\AppData\Local\Programs\Python\Python313\Lib\i
mportlib\__init__.py", line 88, in import_module
    return _bootstrap._gcd_import(name[level:], package, level)

~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^
  File "<frozen importlib._bootstrap>", line 1387, in _gcd_import
  File "<frozen importlib._bootstrap>", line 1360, in _find_and_load
  File "<frozen importlib._bootstrap>", line 1331, in
_find_and_load_unlocked
  File "<frozen importlib._bootstrap>", line 935, in _load_unlocked
  File "<frozen importlib._bootstrap_external>", line 1023, in
exec_module
  File "<frozen importlib._bootstrap>", line 488, in
_call_with_frames_removed
  File "F:\box_Selection\project\urls.py", line 22, in <module>
    path('', include('box_selector.urls')),
           ~~~~~~~^^^^^^^^^^^^^^^^^^^^^^
  File "F:\box_Selection\venv\Lib\site-packages\django\urls\conf.py",
line 39, in include
    urlconf_module = import_module(urlconf_module)
  File
```

```
"C:\Users\Mahajan\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py", line 88, in import_module
    return _bootstrap._gcd_import(name[level:], package, level)

~~~~~~~~~~~~~~~~~~~~~~~~~~^^^
```

```
def home(request):
    products = Product.objects.all().order_by("name")
    selected_product = None
    recommended_box = None
    checked = False
    error = None

    if request.method == "POST":
        checked = True
        product_id = request.POST.get("product_id")

        try:
            selected_product = Product.objects.get(pk=product_id)
            recommended_box = recommend_box(
                selected_product,
                ShippingBox.objects.all()
            )

            if recommended_box is None:
                error = "No suitable box is available for this product."

        except (Product.DoesNotExist, ValueError, TypeError):
            error = "Please select a valid product."

    return render(
        request,
        "box_selector/home.html",
        {
            "products": products,
            "selected_product": selected_product,
            "recommended_box": recommended_box,
            "checked": checked,
            "error": error,
        },
    )
```

Important: Press `Ctrl + S` to save the file. Check that the filename is `views.py`, not `views.py.txt`.

Step 3: Verify that the function exists

In the terminal, run:

```
python -c "import box_selector.views as v; print(v.__file__); print(hasattr(v, 'home'))"
```

The expected output should end with:

```
F:\box_Selection\box_selector\views.py
True
```

If it shows `False` or another error, send me that output.

Step 4: Run the check again

```
python manage.py check
```

If the check passes, start the server:

```
python manage.py runserver
```

If the error continues

Please send me a screenshot of your VS Code Explorer showing the `box_selector` folder and the complete contents of `views.py` . That will help identify why Django is not detecting the function.

You said:

```
return import_module(self.urlconf_name)
  File
"C:\Users\Mahajan\AppData\Local\Programs\Python\Python313\Lib\i
mportlib\__init__.py", line 88, in import_module
    return _bootstrap._gcd_import(name[level:], package, level)

~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^
```

```
  File "<frozen importlib._bootstrap>", line 1387, in _gcd_import
  File "<frozen importlib._bootstrap>", line 1360, in _find_and_load
  File "<frozen importlib._bootstrap>", line 1331, in
_find_and_load_unlocked
  File "<frozen importlib._bootstrap>", line 935, in _load_unlocked
  File "<frozen importlib._bootstrap_external>", line 1023, in
exec_module
  File "<frozen importlib._bootstrap>", line 488, in
_call_with_frames_removed
  File "F:\box_Selection\project\urls.py", line 22, in <module>
    path('', include('box_selector.urls')),
             ~~~~~~~^^^^^^^^^^^^^^^^^^^^^
  File "F:\box_Selection\venv\Lib\site-packages\django\urls\conf.py",
line 39, in include
    urlconf_module = import_module(urlconf_module)
  File
"C:\Users\Mahajan\AppData\Local\Programs\Python\Python313\Lib\i
mportlib\__init__.py", line 88, in import_module
    return _bootstrap._gcd_import(name[level:], package, level)

~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^
  File "<frozen importlib._bootstrap>", line 1387, in _gcd_import
  File "<frozen importlib._bootstrap>", line 1331, in
_find_and_load_unlocked
  File "<frozen importlib._bootstrap>", line 935, in _load_unlocked
  File "<frozen importlib._bootstrap_external>", line 1023, in
exec_module
  File "<frozen importlib._bootstrap>", line 488, in
_call_with_frames_removed
  File "F:\box_Selection\box_selector\urls.py", line 7, in <module>
    path('', views.home, name='home'),
             ^^^^^^^^^^
AttributeError: module 'box_selector.views' has no attribute 'home'
(venv) PS F:\box_Selection> python manage.py check
Traceback (most recent call last):
  File "F:\box_Selection\manage.py", line 22, in <module>
    main()
    ~~~~^^
  File "F:\box_Selection\manage.py", line 18, in main
    execute_from_command_line(sys.argv)
    ~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^
```

```
  File "F:\box_Selection\venv\Lib\site-
packages\django\core\management\__init__.py", line 443, in
execute_from_command_line
    utility.execute()
    ~~~~~~~~~~~~~~~~^^
  File "F:\box_Selection\venv\Lib\site-
packages\django\core\management\__init__.py", line 437, in execute
    self.fetch_command(subcommand).run_from_argv(self.argv)

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^
^^^^^^^^^^
  File "F:\box_Selection\venv\Lib\site-
packages\django\core\management\base.py", line 422, in
run_from_argv
    self.execute(*args, **cmd_options)
    ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^
  File "F:\box_Selection\venv\Lib\site-
packages\django\core\management\base.py", line 466, in execute
    output = self.handle(*args, **options)
  File "F:\box_Selection\venv\Lib\site-
packages\django\core\management\commands\check.py", line 81, in
handle
    self.check(
    ~~~~~~~~~~~^
        app_configs=app_configs,
        ^^^^^^^^^^^^^^^^^^^^^^^^
    ...<4 lines>...
        databases=options["databases"],
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "F:\box_Selection\venv\Lib\site-
packages\django\core\management\base.py", line 498, in check
    all_issues = checks.run_checks(
        app_configs=app_configs,
    ...<2 lines>...
        databases=databases,
    )
  File "F:\box_Selection\venv\Lib\site-
packages\django\core\checks\registry.py", line 99, in run_checks
    new_errors = check(app_configs=app_configs,
databases=databases)
```

```
  File "F:\box_Selection\venv\Lib\site-
packages\django\core\checks\urls.py", line 136, in
check_custom_error_handlers
    handler = resolver.resolve_error_handler(status_code)
  File "F:\box_Selection\venv\Lib\site-
packages\django\urls\resolvers.py", line 743, in resolve_error_handler
    callback = getattr(self.urlconf_module, "handler%s" % view_type,
None)
                    ^^^^^^^^^^^^^^^^^^^^
  File "F:\box_Selection\venv\Lib\site-
packages\django\utils\functional.py", line 47, in __get__
    res = instance.__dict__[self.name] = self.func(instance)
                                ~~~~~~~~~~^^^^^^^^^^^
  File "F:\box_Selection\venv\Lib\site-
packages\django\urls\resolvers.py", line 722, in urlconf_module
    return import_module(self.urlconf_name)
  File
"C:\Users\Mahajan\AppData\Local\Programs\Python\Python313\Lib\i
mportlib\__init__.py", line 88, in import_module
    return _bootstrap._gcd_import(name[level:], package, level)

~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^
  File "<frozen importlib._bootstrap>", line 1387, in _gcd_import
  File "<frozen importlib._bootstrap>", line 1360, in _find_and_load
  File "<frozen importlib._bootstrap>", line 1331, in
_find_and_load_unlocked
  File "<frozen importlib._bootstrap>", line 935, in _load_unlocked
  File "<frozen importlib._bootstrap_external>", line 1023, in
exec_module
  File "<frozen importlib._bootstrap>", line 488, in
_call_with_frames_removed
  File "F:\box_Selection\project\urls.py", line 22, in <module>
    path('', include('box_selector.urls')),
           ~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^
  File "F:\box_Selection\venv\Lib\site-packages\django\urls\conf.py",
line 39, in include
    urlconf_module = import_module(urlconf_module)
  File
"C:\Users\Mahajan\AppData\Local\Programs\Python\Python313\Lib\i
mportlib\__init__.py", line 88, in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
```

```
~~~~~~~~~~~~~~~~~~~~~~~~^^^
```

```
    ~~~~~~~~~~~~^^^^^^
  File "F:\box_Selection\venv\Lib\site-
packages\django\conf\__init__.py", line 102, in _setup
    raise ImproperlyConfigured(
    ...<4 lines>...
    )
django.core.exceptions.ImproperlyConfigured: Requested setting
INSTALLED_APPS, but settings are not configured. You must either
define the environment variable DJANGO_SETTINGS_MODULE or call
settings.configure() before accessing settings.
(venv) PS F:\box_Selection>
```

ChatGPT said:

I checked your error carefully. There are **two separate issues**:

1. Django cannot find the `home()` function in `views.py`.
2. When you tested the import manually, Django settings were not configured. That second error is expected when importing Django models outside Django's setup.

Let's fix this step by step. Don't create a new project or delete anything.

Step 1: Check `settings.py`

Open this file:

```
F:\box_Selection\project\settings.py
```

Find `INSTALLED_APPS` and make sure `'box_selector'` is included:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    'box_selector',
]
```

Save using `Ctrl + S`.

Step 2: Verify `views.py`

Open:

`F:\box_Selection\box_selector\views.py`

Make sure the file contains a function defined exactly as `def home(request):` —with no indentation before `def`.

For now, to isolate the problem, replace the entire file with this **temporary test code**:

```
from django.http import HttpResponse


def home(request):
    return HttpResponse("Box Selection System is working!")
```

Save with `Ctrl + S`.

This temporarily bypasses the models and recommendation code so we can determine whether Django can find the view.

Step 3: Run the check

In your existing terminal, where you see `(venv) PS F:\box_Selection>`, run:

```
python manage.py check
```

If successful, run:

```
python manage.py runserver
```

Open `http://127.0.0.1:8000/` in your browser. You should see:

`Box Selection System is working!`

Step 4: Tell me the result

If the check still fails, copy the **new complete error** here.

If it succeeds, tell me **"working"**. Then we'll restore the actual box recommendation logic in `views.py` without losing your existing code.

You said:

working

ChatGPT said:

Great! 🥕 Your Django project is running now.

Next, we'll restore the **actual Box Selection System logic** so it can recommend the lowest-cost shipping box for a product.

Step 1: Update `views.py`

Open `F:\box_Selection\box_selector\views.py` .

Replace the temporary test code with this complete code:

```
from django.shortcuts import render
from .models import Product, ShippingBox
from .recommendation import recommend_box


def home(request):
    products = Product.objects.all().order_by("name")
    selected_product = None
    recommended_box = None
    checked = False
    error = None

    if request.method == "POST":
        checked = True
        product_id = request.POST.get("product_id")

        try:
```

```
            selected_product = Product.objects.get(pk=product_id)

            recommended_box = recommend_box(
                selected_product,
                ShippingBox.objects.all()
            )

            if recommended_box is None:
                error = "No suitable box is available for this product."

        except (Product.DoesNotExist, ValueError, TypeError):
            error = "Please select a valid product."

    return render(
        request,
        "box_selector/home.html",
        {
            "products": products,
            "selected_product": selected_product,
            "recommended_box": recommended_box,
            "checked": checked,
            "error": error,
        },
    )
```

Save using **Ctrl** + **S**.

Step 2: Check the project

In the terminal, run:

```
python manage.py check
```

If successful, start the server:

```
python manage.py runserver
```

Step 3: Open the website

Visit `http://127.0.0.1:8000/` .

The page should display your box-selection interface, provided your `home.html` template is in the correct location.

Important: If you get a `TemplateDoesNotExist` error, check that the file is located at:

```
F:\box_Selection\box_selector\templates\box_selector\home.html
```

Once the page loads, our next step will be to add sample products and shipping boxes through Django Admin, then test the recommendations.

You said:

give me also process of add and select

ChatGPT said:

हो, आता आपण **Product Add करणे, Shipping Box Add करणे आणि Website वर Product Select करून योग्य Box मिळवणे** हे पूर्ण process step-by-step करूया. 😊

Step 1: Django Admin मध्ये login करा

तुमच्या terminal मध्ये `(venv)` दिसत असेल, तर हा command चालवा:

```
python manage.py createsuperuser
```

Username, email आणि password भरा. नंतर server सुरू करा:

```
python manage.py runserver
```

Browser मध्ये उघडा:

Django Admin
http://127.0.0.1:8000/admin/ ↗

Open

तुमच्या superuser credentials वापरून login करा.

Step 2: Product आणि Shipping Box Admin मध्ये दिसतील याची खात्री करा

जर Admin page वर `Products` आणि `Shipping boxes` दिसत नसतील, तर `box_selector/admin.py` मध्ये हा code ठेवा:

```
from django.contrib import admin
from .models import Product, ShippingBox


@admin.register(Product)
class ProductAdmin(admin.ModelAdmin):
    list_display = ("id", "name", "length", "width", "height", "weight")


@admin.register(ShippingBox)
class ShippingBoxAdmin(admin.ModelAdmin):
    list_display = (
        "id", "name", "length", "width", "height",
        "max_weight", "cost"
    )
```

Save करा आणि Admin page refresh करा.

Step 3: Sample Products Add करा

Admin page वर **Products → Add Product** निवडा. खालीलप्रमाणे माहिती भरा.

Example Product 1

Headphones

Length (cm)	Width (cm)
10	**8**
Height (cm)	Weight (kg)
5	**0.5**

`Save and add another` वापरून दुसरा product add करू शकता.

Product	Length	Width	Height	Weight

Headphones	10 cm	8 cm	5 cm	0.5 kg
Watch	8 cm	6 cm	4 cm	0.2 kg
Shoes	30 cm	12 cm	10 cm	1.0 kg

प्रत्येक product साठी dimensions आणि weight योग्य field मध्ये टाका.

Step 4: Shipping Boxes Add करा

Admin page वर **Shipping Boxes → Add Shipping Box** निवडा.

Example Box 1 — Small Box

Length (cm)
12

Width (cm)
10

Height (cm)
7

Maximum weight (kg)
1

Cost

₹20

आणखी दोन boxes add करा:

Box name	Length	Width	Height	Max weight	Cost
Small Box	12 cm	10 cm	7 cm	1 kg	₹20
Medium Box	20 cm	15 cm	10 cm	5 kg	₹35
Large Box	35 cm	20 cm	15 cm	10 kg	₹50

प्रत्येक box save करा.

Step 5: Website वर Product Select करा

आता website उघडा:

Box Selection Website
http://127.0.0.1:8000/ ↗

Open website

1. Product dropdown मध्ये `Headphones` select करा.
2. **Recommend Box** button वर click करा.
3. Website ने `Small Box` recommend करायला हवा, कारण तो product मध्ये बसतो आणि उपलब्ध suitable boxes पैकी सर्वात कमी किमतीचा आहे.

Expected result

Recommended Box: Small Box ₹20

For Headphones

हे उदाहरणातील dimensions आणि prices वर आधारित अपेक्षित result आहे.

Step 6: वेगवेगळे Products test करा

- **Headphones:** Small Box recommend व्हायला हवा.
- **Watch:** Small Box recommend व्हायला हवा.
- **Shoes:** Medium Box recommend व्हायला हवा, कारण Small Box मध्ये shoes बसत नाहीत.
- **Oversized product:** कोणत्याही box मध्ये बसत नसेल, तर `No suitable box is available for this product.` असा message दिसायला हवा.

महत्त्वाचे: हे selection तेव्हाच काम करेल जेव्हा तुमच्या `home.html` मध्ये product dropdown आणि `Recommend Box` form असेल, आणि `recommendation.py` मध्ये dimension व weight तपासणारा code असेल.

हे steps करून बघा. Admin page वर Products आणि Shipping Boxes दिसतात का, आणि website वर dropdown मध्ये products दिसतात का, ते मला सांगा.

You said:

yevdch ahe ka project github sodun

ChatGPT said:

नाही 😊 GitHub सोडून अजून काही महत्त्वाच्या गोष्टी बाकी आहेत. आपला **Tradexa Technologies चा Django Box Selection Project** पूर्ण करण्यासाठी खालील checklist पूर्ण करूया.

Project मध्ये काय बाकी आहे?

Submission checklist

0/10 complete

- [ ] **Django website working**
  Product select करून योग्य box recommend होतो का ते तपासणे.
- [ ] **Admin मध्ये data add करणे**
  Products आणि Shipping Boxes add करून testing करणे.
- [ ] **Automated test cases**
  python manage.py test box_selector चालवून सर्व tests तपासणे.
- [ ] **README.md**
  Project setup, features आणि run करण्याच्या instructions.
- [ ] **AI_USAGE.md**
  वापरलेले AI prompts, accepted/rejected output आणि verification प्रामाणिकपणे लिहिणे.
- [ ] **TEST_OUTPUT.md**
  Actual test command आणि terminal output नोंदवणे.
- [ ] **Chat transcript**
  तुमच्या actual ChatGPT conversation चा export जोडणे; बनावट transcript नको.
- [ ] **What did you learn?**
  Assignment नुसार स्वतःच्या शब्दांत लिहिणे.
- [ ] **requirements.txt आणि .gitignore**
  Dependencies आणि अनावश्यक files व्यवस्थित ठेवणे.
- [ ] **GitHub repository**
  Code आणि आवश्यक submission files upload करणे.

आधी काय करायचे?

माझ्या मते, आत्ता GitHub ला जाण्यापूर्वी या क्रमाने काम कर:

1. Website वर Product dropdown आणि recommendation व्यवस्थित चालते का तपास.
2. `python manage.py test box_selector` चालव.
3. Test मध्ये errors असतील तर ते fix कर.
4. README, AI_USAGE आणि इतर submission files तयार कर.
5. शेवटी GitHub वर upload कर.

एक महत्त्वाची गोष्ट: Assignment मध्ये actual chat transcript आणि "What did you learn?" स्वतःच्या अनुभवावर आधारित असणे आवश्यक आहे. त्यामुळे ते बनावट तयार करू नकोस.

आधी मला सांग: **तुझ्या website वर Product select करून** `Recommend Box` **केल्यावर result दिसतोय का?** त्यानुसार आपण पुढचा step करू.

You said:

mala aata create `README.md` ithun sagal step by step sang marathi madhun

ChatGPT said:

हो, आपण आता `README.md` **तयार करण्यापासून पुढचे सर्व steps मराठीत, एक-एक करून** करूया. 😊 GitHub वर जाण्याआधी project व्यवस्थित document करू, testing करू आणि मग submission तयार करू.

आपला project: **AI-Assisted Box Selection System — Django**

Step 1: `README.md` फाइल कुठे तयार करायची?

तुझा project folder हा आहे:

```
F:\box_Selection\
```

`README.md` फाइल ही `manage.py` **ज्या folder मध्ये आहे, त्याच मुख्य project folder मध्ये** तयार करायची. `box_selector` च्या आत नाही.

box_Selection
- `manage.py`
- `project/`
- `box_selector/`
- **README.md ← इथे तयार करायची**
- `requirements.txt` (पुढे तयार करू)

VS Code मध्ये तयार करण्याची पद्धत

1. VS Code च्या डाव्या बाजूला **Explorer** उघड.

2. सर्वात वरच्या `box_Selection` folder वर right-click कर.
3. **New File** निवड.
4. फाइलचे नाव अचूक लिही: `README.md`
5. Enter दाब.

`README.md.txt` असे नाव होणार नाही याची खात्री कर.

Step 2: `README.md` मध्ये हा content टाक

`README.md` फाइल उघड आणि खालील content paste कर. नंतर **Ctrl + S** दाबून save कर.

Writing

```
# AI-Assisted Box Selection System

## 1. Project Overview

The AI-Assisted Box Selection System is a Django web application that recommends a suitable shipping box for a product.

The system checks the product's length, width, height, and weight against the available shipping boxes. It selects the lowest-cost box that satisfies the dimension and weight requirements.

**Note:** The current implementation uses rule-based recommendation logic. It does not use a trained machine learning model.

## 2. Objectives

- Store product dimensions and weight.
- Store shipping box dimensions, maximum weight capacity, and cost.
- Recommend a suitable box for a selected product.
- Choose the lowest-cost suitable box.
- Display a message when no suitable box is available.
- Manage products and boxes through Django Admin.

## 3. Technologies Used
```

- Python
- Django
- SQLite
- HTML
- Django Template Language

4. Features

- Product management through Django Admin.
- Shipping box management through Django Admin.
- Product selection from a dropdown list.
- Dimension and weight validation.
- Lowest-cost suitable box recommendation.
- Handles products that do not fit any available box.

5. Project Structure

```text
box_Selection/
├── manage.py
├── project/
│   ├── settings.py
│   └── urls.py
├── box_selector/
│   ├── migrations/
│   ├── templates/
│   │   └── box_selector/
│   │       └── home.html
│   ├── admin.py
│   ├── models.py
│   ├── recommendation.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py
├── README.md
```

```
├── requirements.txt
├── AI_USAGE.md
└── TEST_OUTPUT.md
```

6. Setup and Installation

Prerequisites

- Python installed
- pip installed
- Visual Studio Code or another code editor

Step 1: Open the project directory

Open the terminal in the project folder.

Step 2: Create a virtual environment

```bash
python -m venv venv
```

Step 3: Activate the virtual environment

For Windows PowerShell:

```powershell
.\venv\Scripts\Activate.ps1
```

Step 4: Install dependencies

```bash
python -m pip install -r requirements.txt
```

Step 5: Apply database migrations

```bash
python manage.py migrate
```

Step 6: Create an admin account

```bash
python manage.py createsuperuser
```

Follow the terminal prompts to create the account.

Step 7: Start the development server

```bash
python manage.py runserver
```

Open the application:

http://127.0.0.1:8000/

Open Django Admin:

http://127.0.0.1:8000/admin/

7. How to Use the Application

1. Start the Django development server.
2. Open Django Admin.
3. Add products with their dimensions and weight.
4. Add shipping boxes with their dimensions, maximum weight capacity, and cost.
5. Open the main application page.

6. Select a product from the dropdown.
7. Click the Recommend Box button.
8. Review the recommended box or the message indicating that no suitable box is available.

8. Recommendation Logic

The application sorts the three product dimensions and the three box dimensions before comparing them. This allows the product to fit in a different orientation.

A box is considered suitable when:

- All three product dimensions fit within the corresponding box dimensions.
- The product weight does not exceed the box's maximum weight capacity.

Among suitable boxes, the application selects the box with the lowest cost.

If no box satisfies both conditions, the application returns no recommendation.

9. Running Tests

Run the automated tests using:

```bash
python manage.py test box_selector
```

The tests cover dimension fitting, weight capacity, lowest-cost selection, rotation, exact limits, and the no-suitable-box case.

Record the actual test results in `TEST_OUTPUT.md`.

10. AI Usage and Learning

AI tool usage, prompts, accepted or rejected suggestions, errors encountered, and verification steps are documented in `AI_USAGE.md`.

The learning reflection should describe my own understanding and experience while building and testing this project.

11. Future Improvements

- Add support for multiple products in a single order.
- Add more advanced packing logic for products with complex shapes.
- Improve the user interface.
- Add further validation and test cases.

12. Project Status

This project is intended as a Django hiring assignment. Features and test results should be verified before submission.

महत्त्वाची टीप: README मधील project structure आणि features तुझ्या actual files शी जुळतात याची खात्री कर. `requirements.txt`, `AI_USAGE.md` आणि `TEST_OUTPUT.md` अजून तयार नसतील तर काळजी करू नकोस—ते आपण पुढे तयार करू.

Step 3: आता `requirements.txt` तयार कर

VS Code terminal मध्ये खात्री कर की तू `F:\box_Selection` मध्ये आहेस आणि `(venv)` active आहे. मग हे चालव:

```
python -m pip freeze > requirements.txt
```

यामुळे तुझ्या virtual environment मधील installed Python packages ची यादी तयार होईल.

तपासण्यासाठी:

```
Get-Content requirements.txt
```

यात Django आणि इतर installed packages दिसायला हवेत.

Step 4: `README.md` save झाली का तपास

आता पुढील तीन गोष्टी कर:

- [ ] `README.md` मुख्य `box_Selection` folder मध्ये तयार केली.
- [ ] README चा content paste करून `Ctrl + S` केले.
- [ ] `requirements.txt` तयार केला.

यानंतर आपण `AI_USAGE.md` तयार करू. त्यात AI चा वापर कसा केला, कोणत्या सूचना स्वीकारल्या, कोणत्या चुका आल्या आणि स्वतः verify कसे केले हे **खऱ्या अनुभवावर आधारित** लिहायचे आहे.

If you want, I can:

- README.md चे पुढील भाग लिहा आणि project setup समजून घ्या
- requirements.txt तयार करण्याची प्रक्रिया मागवा
- AI_USAGE.md आणि TEST_OUTPUT.md फाइल्स तयार करण्यासाठी मार्गदर्शन मागवा